

CS 310 : Automata Theory

S. Krishna

Automata Theory

What is this course about?

- ▶ Mathematical models for computation and computers

Automata Theory

What is this course about?

- ▶ Mathematical models for computation and computers
- ▶ Limits of computers and computation

Automata Theory

What is this course about?

- ▶ Mathematical models for computation and computers
- ▶ Limits of computers and computation
 - ▶ Can a computer do ANYTHING?
 - ▶ Are there things that cannot be solved?

Course Outline

What we plan to cover

1. Finite automata and regular languages/expressions and more...
2. Pushdown automata, context-free grammars and other models of computation
3. Turing machines and computability: the extents and limits of computation
4. Efficient computation: basic complexity classes...

Logistics

Evaluation

- ▶ Quizzes 30%
- ▶ Midsem 25%
- ▶ Endsem 40%
- ▶ Pop quizzes 5%
- ▶ Quiz dates : August 24, October 12, October 26 (as per CRs)

Logistics

Evaluation

- ▶ Quizzes 30%
- ▶ Midsem 25%
- ▶ Endsem 40%
- ▶ Pop quizzes 5%
- ▶ Quiz dates : August 24, October 12, October 26 (as per CRs)

Recommended text

- ▶ Intro to Automata Theory, Languages and Computation: Hopcroft, Motwani, Ullman
- ▶ Intro to Theory of Computation: Michael Sipser
- ▶ Automata and Computability : Kozen
- ▶ More references as we proceed

Logistics (Contd.)

Course material

On Piazza

Discussions

- ▶ For online discussions: Piazza
- ▶ Weekly problem solving session: 1 hour per week (To be fixed by CRs and TAs).
- ▶ Office hours: Thursday 16.00-17.00 hrs

Recap

As part of CS 228 we have already seen

- ▶ DFAs, NFAs, ϵ -NFAs : all capturing regular languages

Recap

As part of CS 228 we have already seen

- ▶ DFAs, NFAs, ϵ -NFAs : all capturing regular languages
- ▶ Closure of regular languages under union, intersection and complementation

Recap

As part of CS 228 we have already seen

- ▶ DFAs, NFAs, ϵ -NFAs : all capturing regular languages
- ▶ Closure of regular languages under union, intersection and complementation
- ▶ DFAs can incur an exponential blowup in size compared to the corresponding **equivalent** NFAs, ϵ -NFAs
- ▶ Two machines are equivalent if they accept the same language

Recap

As part of CS 228 we have already seen

- ▶ DFAs, NFAs, ϵ -NFAs : all capturing regular languages
- ▶ Closure of regular languages under union, intersection and complementation
- ▶ DFAs can incur an exponential blowup in size compared to the corresponding **equivalent** NFAs, ϵ -NFAs
- ▶ Two machines are equivalent if they accept the same language

Last class : question from Sayan

What if we modify the accepting condition

accept a word

only if

all the runs end in final states?

Does it still recognize only regular languages?

Recap : Angelic and Devilish Acceptance

- ▶ Standard acceptance (angelic) in NFAs : at least one run is accepting
- ▶ Devilish acceptance condition : all runs of a word w in the NFA must end in an accepting state

Recap : Angelic and Devilish Acceptance

- ▶ Standard acceptance (angelic) in NFAs : at least one run is accepting
- ▶ Devilish acceptance condition : all runs of a word w in the NFA must end in an accepting state
- ▶ To show equivalence, show that the devilish condition in NFAs also captures regular languages

Recap : Angelic and Devilish Acceptance

- ▶ Standard acceptance (angelic) in NFAs : at least one run is accepting
- ▶ Devilish acceptance condition : all runs of a word w in the NFA must end in an accepting state
- ▶ To show equivalence, show that the devilish condition in NFAs also captures regular languages
- ▶ Convert an NFA with devilish acceptance into an equivalent DFA

An Algebraic View of Regular Languages

Arithmetic Expressions

- ▶ $(2+3)-5$
- ▶ $23 \times 5 - 4$
- ▶ Expressions combine numbers with arithmetic operations, the value of such an expression is a number
- ▶ Value of $(2+3)-5$ is 0 and so on

Arithmetic Expressions

- ▶ $(2+3)-5$
- ▶ $23 \times 5 - 4$
- ▶ Expressions combine numbers with arithmetic operations, the value of such an expression is a number
- ▶ Value of $(2+3)-5$ is 0 and so on
- ▶ Can we have “regular expressions” whose value is a regular language?

Combining Base Expressions

Base Expressions

- ▶ The empty word ϵ . Its value $L(\epsilon)$ is the regular language $\{\epsilon\}$.
- ▶ The empty set $\emptyset$. Its value $L(\emptyset)$ is the regular language $\{\}$.
- ▶ Any symbol a from an alphabet Σ . Its value $L(a)$ is the regular language $\{a\}$.

Combining Base Expressions

Base Expressions

- ▶ The empty word ϵ . Its value $L(\epsilon)$ is the regular language $\{\epsilon\}$.
- ▶ The empty set $\emptyset$. Its value $L(\emptyset)$ is the regular language $\{\}$.
- ▶ Any symbol a from an alphabet Σ . Its value $L(a)$ is the regular language $\{a\}$.

Operations

- ▶ Concatenation
- ▶ Union
- ▶ Kleene Star

Combining Base Expressions

Operations on Languages

- ▶ Concatenation $L_1 \cdot L_2 = \{w_1 \cdot w_2 \mid w_1 \in L_1, w_2 \in L_2\}$
- ▶ Union $L_1 \cup L_2 = \{w \mid w \in L_1 \text{ or } w \in L_2\}$
- ▶ Kleene Star : For a language L , its **Kleene closure**, denoted L^* is the set of all strings obtained by taking any number of strings from L with possible repetitions and concatenating all of them.

Combining Base Expressions

Operations on Languages

- ▶ Concatenation $L_1 \cdot L_2 = \{w_1 \cdot w_2 \mid w_1 \in L_1, w_2 \in L_2\}$
- ▶ Union $L_1 \cup L_2 = \{w \mid w \in L_1 \text{ or } w \in L_2\}$
- ▶ Kleene Star : For a language L , its **Kleene closure**, denoted L^* is the set of all strings obtained by taking any number of strings from L with possible repetitions and concatenating all of them.

$$L^* = \cup_{i \geq 0} L^i$$

where $L^0 = \{\epsilon\}$, $L^1 = L$, L^i for $i > 1 = L \circ L^{i-1}$.

Combining Base Expressions

Operations on Expressions

- ▶ Given expressions R_1 and R_2 , $(R_1 \cdot R_2)$ is an expression, with value $L(R) = L(R_1) \cdot L(R_2)$,
- ▶ Given expressions R_1 and R_2 , $(R_1 \cup R_2)$ (or $(R_1 + R_2)$) is an expression, with value $L(R) = L(R_1) \cup L(R_2)$,
- ▶ For an expression R , $(R)^*$ is an expression with value $L(R^*) = [L(R)]^*$.

Regular Expressions : An Inductive Definition

Say that R is a regular expression if R is

- ▶ $a \in \Sigma, \epsilon$ or $\emptyset$,
- ▶ $(R_1 \cup R_2)$ for regular expressions R_1, R_2
- ▶ $(R_1 \cdot R_2)$ for regular expressions R_1, R_2
- ▶ $(R_1)^*$ for some regular expression R_1

We omit $\cdot$ when clear and write $R_1 R_2$ instead of $R_1 \cdot R_2$. In the absence of paranthesis, Kleene star binds tightest, followed by concatenation followed by union.

Equivalence

Regular expressions R_1, R_2 are called **equivalent** if $L(R_1) = L(R_2)$

Examples

Assume $\Sigma = \{a, b\}$. What is $L(R)$ for the following R ?

► a^*ba^*

Examples

Assume $\Sigma = \{a, b\}$. What is $L(R)$ for the following R ?

- ▶ a^*ba^*
- ▶ $(\Sigma^*a\Sigma^*) \cup (\Sigma^*b\Sigma^*)$

Examples

Assume $\Sigma = \{a, b\}$. What is $L(R)$ for the following R ?

- ▶ a^*ba^*
- ▶ $(\Sigma^*a\Sigma^*) \cup (\Sigma^*b\Sigma^*)$
- ▶ $(\Sigma^*a\Sigma^*b\Sigma^*) \cup (\Sigma^*b\Sigma^*a\Sigma^*)$

Examples

Assume $\Sigma = \{a, b\}$. What is $L(R)$ for the following R ?

- ▶ a^*ba^*
- ▶ $(\Sigma^*a\Sigma^*) \cup (\Sigma^*b\Sigma^*)$
- ▶ $(\Sigma^*a\Sigma^*b\Sigma^*) \cup (\Sigma^*b\Sigma^*a\Sigma^*)$
- ▶ $(\Sigma\Sigma)^*$

Examples

Assume $\Sigma = \{a, b\}$. What is $L(R)$ for the following R ?

- ▶ a^*ba^*
- ▶ $(\Sigma^*a\Sigma^*) \cup (\Sigma^*b\Sigma^*)$
- ▶ $(\Sigma^*a\Sigma^*b\Sigma^*) \cup (\Sigma^*b\Sigma^*a\Sigma^*)$
- ▶ $(\Sigma\Sigma)^*$
- ▶ $(a \cup \epsilon)b^*$

Examples

Assume $\Sigma = \{a, b\}$. What is $L(R)$ for the following R ?

- ▶ a^*ba^*
- ▶ $(\Sigma^*a\Sigma^*) \cup (\Sigma^*b\Sigma^*)$
- ▶ $(\Sigma^*a\Sigma^*b\Sigma^*) \cup (\Sigma^*b\Sigma^*a\Sigma^*)$
- ▶ $(\Sigma\Sigma)^*$
- ▶ $(a \cup \epsilon)b^*$
- ▶ $(a \cup \epsilon)(b \cup \epsilon)$

Examples

Assume $\Sigma = \{a, b\}$. What is $L(R)$ for the following R ?

- ▶ a^*ba^*
- ▶ $(\Sigma^*a\Sigma^*) \cup (\Sigma^*b\Sigma^*)$
- ▶ $(\Sigma^*a\Sigma^*b\Sigma^*) \cup (\Sigma^*b\Sigma^*a\Sigma^*)$
- ▶ $(\Sigma\Sigma)^*$
- ▶ $(a \cup \epsilon)b^*$
- ▶ $(a \cup \epsilon)(b \cup \epsilon)$
- ▶ $a^*\emptyset$

Examples

Assume $\Sigma = \{a, b\}$. What is $L(R)$ for the following R ?

- ▶ a^*ba^*
- ▶ $(\Sigma^*a\Sigma^*) \cup (\Sigma^*b\Sigma^*)$
- ▶ $(\Sigma^*a\Sigma^*b\Sigma^*) \cup (\Sigma^*b\Sigma^*a\Sigma^*)$
- ▶ $(\Sigma\Sigma)^*$
- ▶ $(a \cup \epsilon)b^*$
- ▶ $(a \cup \epsilon)(b \cup \epsilon)$
- ▶ $a^*\emptyset$
- ▶ $\emptyset^*a$

Are regular expressions unique?

Simplify $1^*01 + 1^*01(0 + \epsilon + 11)^*(0 + \epsilon + 11)$

Are regular expressions unique?

Simplify $1^*01 + 1^*01(0 + \epsilon + 11)^*(0 + \epsilon + 11)$

- ▶ $R_1 + R_1R^*R = R_1(\epsilon + R^*R)$ where $R_1 = 1^*01, R = 0 + \epsilon + 11$
- ▶ $R_1(\epsilon + R^+) = R_1R^* = 1^*01(0 + \epsilon + 11)^*$

Regular Expressions and Regular Languages

Theorem

Regular languages coincide with the languages of regular expressions

Regular expressions to regular languages

- ▶ Show the base expressions give regular languages
- ▶ Recap closure of regular languages under concatenation, union and Kleene-star
- ▶ Done on the board by Nilkrishna and Anshul.

Regular Languages to Regular Expressions

- ▶ Let L be a regular language defined by a DFA A , $L = L(A)$.
- ▶ Compute the regular expression R such that $L(R) = L(A)$.
- ▶ We work with generalized non-deterministic finite automata (GNFA)